

# Discussion sur la terminaison d'une thread

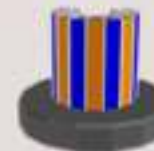
Fabrice.Kordon@lip6.fr

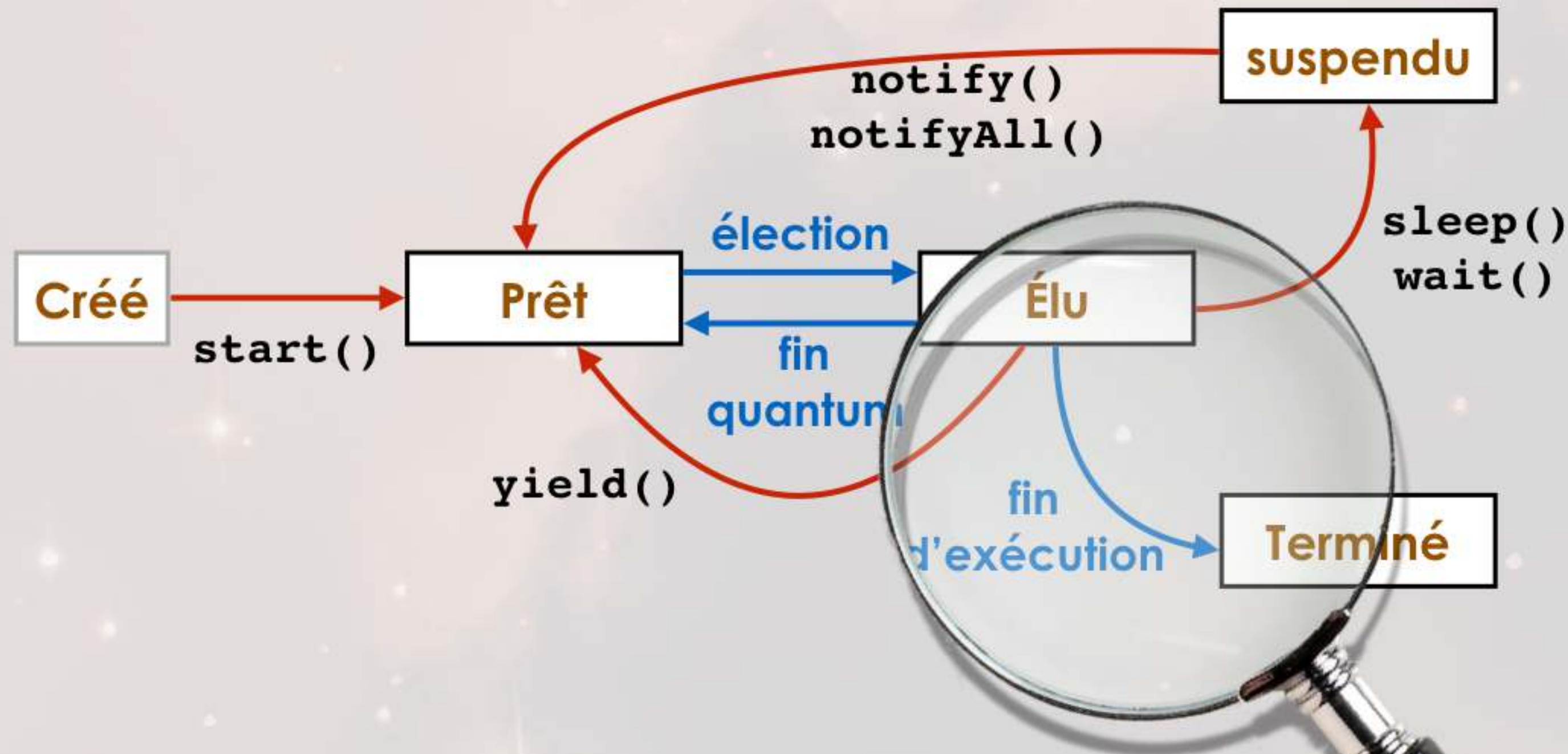




# En guise d'introduction

2

 Vous souvenez-vous?



 Mais comment cela se passe-t-il?

-  Dans un système d'exploitation
-  En Java



# Fonctionnement

3

## À la POSIX : pthread\_create / pthread\_cancel

- 🔧 Géré par le système d'exploitation
- 🔧 Prise en compte de «point d'annulation»
  - ▶ Maintenir la cohérence du système
  - ▶ Éviter de tuer une thread «au mauvais moment» (mutex)

## À la Java

- 🔧 Abandon du stop() car non déterministe
- 🔧 Interrupt() positionne un «flag»
- 🔧 Approche recommandée
  - ▶ Tester explicitement ce «flag» lorsqu'on peut se terminer
  - ▶ Maîtrise explicite de la terminaison par le développeur (et non l'OS)



le «flag» est détruit lors du test



# Exemple (variante d'un exemple déjà vu)

4

```
class Traitement implements Runnable {
    String monNom;
    Thread maThread;

    Traitement (String n) {
        monNom = n;
    }

    public void run() {
        while (true) {
            System.out.println(monNom + " dit coucou");
        }
    }
}

public class AbortThread2 {
    public static void main(String []args) {
        Thread t1, t2;
        Traitement tr1, tr2;

        tr1 = new Traitement("TR1");
        tr2 = new Traitement("TR2");
        t1 = new Thread (tr1);
        t2 = new Thread (tr2);
        t1.start();
        t2.start();
        t1.interrupt();
        System.out.println("J'ai tué tout le monde");
    }
}
```



# Exemple (variante d'un exemple déjà vu)

4

```
class Traitement implements Runnable {
```

```
$ java AbortThread2
```

```
TR1 dit coucou
```

```
TR1 dit coucou
```

```
TR1 dit coucou
```

```
TR1 dit coucou
```

```
TR1 dit coucou
```

```
TR1 dit coucou
```

```
TR2 dit coucou
```

```
TR2 dit coucou
```

```
TR1 dit coucou
```

```
TR1 dit coucou
```

```
TR2 dit coucou
```

```
TR2 dit coucou
```

```
TR1 dit coucou
```

```
TR2 dit coucou
```

```
TR2 dit coucou
```

```
TR1 dit coucou
```

```
TR1 dit coucou
```

```
TR1 dit coucou
```

```
TR1 dit coucou
```

```
^C
```

## Remarque

Pas de destruction pour «TR1»

## Pourquoi?

`sleep()` notifie de l'interruption  
comme d'autres opérations

## Donc...

Il faut faire le test explicitement



# La «bonne manière» (avec interrupt)

5

```
class Traitement implements Runnable {
    String monNom;
    Thread maThread;

    Traitement (String n) {
        monNom = n;
    }

    public void run() {
        while (!Thread.interrupted()) {
            System.out.println(monNom + " dit coucou");
        }
    }
}

public class AbortThread2 {
    public static void main(String []args) {
        Thread t1, t2;
        Traitement tr1, tr2;

        tr1 = new Traitement("TR1");
        tr2 = new Traitement("TR2");
        t1 = new Thread (tr1);
        t2 = new Thread (tr2);
        t1.start();
        t2.start();
        t1.interrupt();
        System.out.println("J'ai tué tout le monde");
    }
}
```



# La «bonne manière» (avec interrupt)

5

```
class Traitement implements Runnable {
```

```
$ java AbortThread  
J'ai tué tout le monde  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
TR2 dit coucou  
^C
```

**Et maintenant?**

TR1 est bien morte (au «bon» moment)



**Rappel, consultation destructive**

Tester le «flag» = reset dans la foulée



# En guise de conclusion...

6

## L'usage de `interrupt()` est délicat

- Il ne faut pas laisser le programme dans un état incohérent
- Il faut contrôler «quand» meurent les threads

## Deux approches

- ▶ L'environnement s'en charge (POSIX)
- ▶ Le développeur s'en charge (Java)

## Important

- les opérations bloquantes renvoie l'exception
  - ▶ Car l'usager ne peut tester (thread bloquée)
- Le status des E/S est un peu particulier
  - ▶ Comportement délicat (RTFM)